Real-Time Web Communication Standards: A Retrospective and Projection 2021–2031

March 2026

Contents

1 Introduction **2**

2 WebRTC **2**
2.1 Late 2021 2
2.2 March 2026 3

3 QUIC **4**
3.1 Late 2021 4
3.2 March 2026 4

4 HTTP/3 **4**
4.1 Late 2021 5
4.2 March 2026 5

5 WebTransport **5**
5.1 Late 2021 5
5.2 March 2026 6

6 WebCodecs **6**
6.1 Late 2021 6
6.2 March 2026 7

7 Summary **7**

8 Conclusion **7**

9 Looking Ahead: 2028–2029 (Two to Three Years) **8**
9.1 WebCodecs Reaches W3C Recommendation 8
9.2 WebTransport Achieves Universal Browser Support 8
9.3 WebRTC Encoded Transforms Become Cross-Browser 9
9.4 AV1 Becomes a Practical WebRTC Codec 9
9.5 Media over QUIC Reaches RFC Status 9
9.6 RTP over QUIC Reaches RFC Status 9

9.7 Hardware and Network Context . . . 9

10 Looking Ahead: 2029–2031 (Three to Five Years) 10
10.1 The "Unbundled" Architecture Becomes Viable . . . 11
10.2 MoQ Enables Standards-Based Low-Latency Streaming . . . 11
10.3 AV1 as the Default WebRTC Codec . . . 12
10.4 QUIC v2 Transparent Adoption . . . 12
10.5 No WebRTC 2.0 . . . 12
10.6 Hardware and Network Context . . . 12

A Information Theoretic View 13
A.1 Channel Capacity of the Transport Layer . . . 14
A.1.1 Head-of-Line Blocking and Multiplexed Capacity . . . 14
A.1.2 Connection Establishment Latency . . . 14
A.2 Source Coding Efficiency . . . 15
A.2.1 Codec Entropy and WebCodecs . . . 15
A.3 Effective Information Rate by Time Horizon . . . 16
A.4 Security Entropy and Key Space . . . 16
A.4.1 End-to-End Encryption via Encoded Transforms . . . 17
A.5 WebTransport: Unreliable Datagrams and Channel Capacity . . . 17
A.6 Projections Summary . . . 18

List of Figures

1 By 2028: infrastructure enables standards to deliver measurable user experience gains. Arrows show enabling relationships. . . . 10
2 Monolithic WebRTC (left) vs. the unbundled architecture (right). The unbundled approach gives applications control over codec, transport, encryption, and congestion decisions that `RTCPeerConnection` handles internally. . . . 11
3 Convergence of standards maturity, hardware capability, and network bandwidth from 2021 to 2031. The shaded region marks where all three curves reach levels sufficient to fully exploit the standards discussed in this document. . . . 13

List of Tables

1 Browser capability status by standard, late 2021 vs. March 2026. . . . 8
2 Effective information rate for a 720p/30fps WebRTC video stream. The declining ratio $I_{\text{net}}/C_{\text{link}}$ reflects both improved codec efficiency and growing link capacity, freeing bandwidth for higher resolutions, additional streams, or concurrent data channels. 16
3 Information-theoretic measures across time horizons. Δ is the coding efficiency gap (11), I_{net} is the net information rate (17), and λ^* is the freshness decay threshold above which WebTransport dominates WebSocket (28). . . . 18

1 Introduction

This document examines the concrete browser capabilities enabled by the real-time web communication standards collected in the `standards/` directory of this project. It compares what these standards made possible in browsers as of late 2021 with what they enable as of March 2026. The discussion focuses exclusively on what standards adoption in browsers enabled — not on market trends, product adoption, or commercial developments.

The standards under review span five areas:

- **WebRTC** — W3C WebRTC 1.0 and supporting IETF protocols (RFCs 8829, 8445, 8826, 8827, 3550, 3711)
- **QUIC** — RFC 9000
- **HTTP/3** — RFC 9114
- **WebTransport** — W3C Working Draft and RFC 9297
- **WebCodecs** — W3C Working Draft

Together, these standards define a full stack for real-time communication in the browser: from low-level transport (QUIC) through application-layer protocols (HTTP/3, WebTransport) to peer-to-peer media and data exchange (WebRTC) and direct codec access (WebCodecs).

2 WebRTC

The WebRTC stack in the `standards/` directory comprises the W3C WebRTC 1.0 specification and six IETF RFCs: JSEP (RFC 8829), ICE (RFC 8445), the WebRTC security considerations (RFC 8826), the DTLS-based security architecture (RFC 8827), RTP (RFC 3550), and SRTP (RFC 3711).

2.1 Late 2021

W3C WebRTC 1.0 achieved Recommendation status in January 2021, making it a finalized standard. By late 2021, Chrome and Firefox offered full implementations, and Edge inherited Chrome's support via its Chromium base. Safari supported the core API but with significant limitations:

- **Codec restrictions.** Safari supported only H.264 for video. Chrome and Firefox supported VP8, VP9, and H.264. This meant Safari could not participate in VP9-based calls and lacked access to VP9's more efficient bandwidth usage.
- **No simulcast or SVC.** Safari could not send multiple encodings of the same video stream at different resolutions (simulcast) or use Scalable Video Coding (SVC). Chrome and Firefox supported both, enabling adaptive quality based on network conditions.
- **RTCDataChannel issues on iOS.** The data channel did not work reliably on iOS Safari. Cross-browser data channel connections between Safari on iOS and Chrome or Firefox failed due to missing local ICE candidates on iOS.
- **SDP handling divergence.** Chrome was in the process of migrating from Plan B to Unified Plan SDP semantics. Chrome M89 (February 2021) added deprecation warnings; Chrome M93 (August 2021) removed Plan B with a reverse origin trial option. Firefox and

Safari already used Unified Plan exclusively. During this transition, applications had to accommodate both SDP formats.

- **Insertable Streams.** Chrome M94 introduced the Insertable Streams API, enabling JavaScript to access and transform encoded media frames in the WebRTC pipeline. This was essential for end-to-end encryption of media. Firefox and Safari did not implement it.
- **Stats reporting inconsistencies.** Firefox was missing `voiceActivityFlag` on video track stats. Safari was missing `rtpTimestamp` and `voiceActivityFlag`. Applications could not reliably detect voice activity or obtain precise timing information across browsers.

On the IETF side, the supporting protocols were in the following state:

- **JSEP (RFC 8829)** was published in January 2021 alongside the W3C Recommendation. All browsers implemented the JSEP signaling state machine through `RTCPeerConnection`, but diverged in areas like BUNDLE handling of multiple media streams.
- **ICE (RFC 8445)** was implemented in all browsers. Trickle ICE (RFC 8838), enabling incremental candidate exchange rather than waiting for full gathering, was published in January 2021 but not yet mandatory.
- **DTLS-SRTP (RFCs 8827, 3711)** was mandatory across all browsers. All media was encrypted. However, browsers differed in DTLS certificate algorithms (Chrome 52+ used ECDSA; others varied), and privacy identifier reset mechanisms were inconsistent.
- **Security (RFC 8826)** — all browsers enforced camera/microphone permission prompts and encrypted all media. However, protections against IP address leakage were inconsistent; not all browsers masked local IP addresses via mDNS candidates when media permissions had not been granted.

2.2 March 2026

The W3C updated the WebRTC 1.0 Recommendation in October 2024 and again in 2025, incorporating amendments based on implementation experience.

- **Unified Plan is universal.** Plan B SDP is completely removed from all browsers. Applications no longer need to accommodate both formats.
- **Safari gained VP9.** Safari 15 and later support the VP9 codec for WebRTC, enabling Safari to participate in VP9-based calls and benefit from VP9's bandwidth efficiency. Safari still prefers H.264 for hardware acceleration, and its SVC support remains limited compared to Chrome and Firefox.
- **iOS WebRTC matured.** Starting with iOS 14.3, WebRTC works reliably across all browsers on iOS. RTCDataChannel connections between iOS Safari and desktop Chrome or Firefox are reliable.
- **JSEP updated to RFC 9429.** Published in April 2024, RFC 9429 obsoleted RFC 8829, addressing implementation divergence — particularly in BUNDLE handling and bundle-only `m=` sections.
- **Trickle ICE is mandatory.** All browsers implement RFC 8838, enabling faster connection establishment through incremental candidate exchange.

- **mDNS privacy protections standardized.** Chrome, Firefox, and Safari all mask local IP addresses using mDNS candidates when camera/microphone permissions have not been granted, implementing the privacy considerations of RFC 8826.
- **Encoded Transforms remain Chrome-only.** The feature evolved from "Insertable Streams" to `RTCRtpScriptTransform` (WebRTC Encoded Transform), but Firefox and Safari still do not implement it. End-to-end encryption of encoded media frames in the WebRTC pipeline remains a Chrome/Chromium-only capability.

3 QUIC

RFC 9000 defines the QUIC transport protocol: a UDP-based, multiplexed, encrypted transport with built-in TLS 1.3, independent stream flow control, and connection migration across network changes.

3.1 Late 2021

RFC 9000 was published in May 2021. Browser support was uneven:

- **Chrome** had QUIC enabled by default, having been an early adopter (Google developed the original QUIC protocol).
- **Firefox 88** (April 2021) enabled QUIC by default.
- **Safari** had experimental QUIC support (added in Safari Technology Preview 104, April 2020), but it required manual opt-in through experimental settings. It was not available to ordinary users.

In practice, a browser connecting to a QUIC-capable server would benefit from reduced connection latency (0-RTT or 1-RTT handshakes vs. TCP's multi-round handshake plus TLS), elimination of head-of-line blocking across multiplexed streams, and connection survival across network changes. But with Safari requiring manual opt-in, these benefits were unavailable to most Safari users.

3.2 March 2026

QUIC is enabled by default in all major browsers:

- Chrome, Edge, and Firefox continue default support.
- Safari 16 (2022) enabled QUIC by default, bringing universal coverage.

Every major browser can now establish QUIC connections transparently. The capabilities defined by RFC 9000 — low-latency connection establishment, multiplexed streams without head-of-line blocking, built-in encryption, and connection migration — are universally available to web applications without any user or developer opt-in.

4 HTTP/3

RFC 9114 maps HTTP semantics over QUIC, inheriting QUIC's transport properties: multiplexed streams without head-of-line blocking, lower connection latency, and connection migration.

4.1 Late 2021

RFC 9114 had not yet been published (it was finalized in June 2022). Browsers were shipping pre-standard HTTP/3 implementations based on draft specifications:

- **Chrome** had experimental HTTP/3 support enabled by default since April 2020.
- **Firefox 88** (April 2021) enabled HTTP/3 by default.
- **Safari** had HTTP/3 support behind an experimental feature flag, disabled by default.

Where supported, browsers could negotiate HTTP/3 with compatible servers, gaining per-stream independence (a stalled stream would not block other streams, unlike HTTP/2 over TCP). However, the lack of a finalized RFC meant implementations were based on draft specifications that could diverge, and Safari's experimental-only status limited universal availability.

4.2 March 2026

With RFC 9114 finalized in June 2022, all major browsers now support HTTP/3 by default:

- Chrome, Edge, and Firefox have supported HTTP/3 by default for several years.
- Safari enabled HTTP/3 by default starting with Safari 16 (2022).
- Over 92% of browser market share supports HTTP/3.

The capabilities defined by RFC 9114 — HTTP request/response multiplexing without head-of-line blocking, lower latency connection establishment via QUIC, and connection resilience across network changes — are now universally available. Any browser connecting to an HTTP/3-capable server automatically benefits from these properties.

5 WebTransport

The WebTransport specification defines browser APIs for bidirectional client-server communication over HTTP/3, supporting both reliable streams and unreliable datagrams. RFC 9297 defines the HTTP datagram and capsule protocol mechanisms that underpin it.

5.1 Late 2021

WebTransport was in a very early state:

- **Chrome** had experimental support but it was disabled in Chrome M91 through M94. The API was not usable in production.
- **Firefox** had no support whatsoever.
- **Safari** had no support whatsoever.
- RFC 9297 had not yet been published (August 2022).
- The W3C specification was an early Working Draft.

In late 2021, WebTransport was effectively unavailable in any browser for production use. The capabilities it defines — unreliable datagrams from browser to server, multiple independent reliable streams, and connection migration — could not be used by web applications.

5.2 March 2026

WebTransport has achieved broad but not universal support:

- **Chrome 97+** and **Edge 98+** support WebTransport (stable since late 2021 / early 2022).
- **Firefox 114+** (2023) supports WebTransport.
- **Safari does not support WebTransport.**
- Approximately 82% of browser market share has WebTransport support.

For browsers that support it, WebTransport enables capabilities that were previously impossible without WebRTC's peer-to-peer architecture:

- **Unreliable datagrams** from browser to server, suitable for latency-sensitive data where retransmission is undesirable (e.g., game state updates, real-time sensor data).
- **Multiple independent reliable streams** without head-of-line blocking between them, enabling parallel data flows over a single connection.
- **Connection migration** across network changes, inherited from QUIC.

Safari's absence means applications using WebTransport must provide fallback mechanisms (typically WebSocket) for Safari users, or accept that these capabilities are unavailable on Apple platforms.

6 WebCodecs

The WebCodecs specification provides JavaScript interfaces for direct access to the browser's built-in audio, video, and image codecs, enabling frame-level encoding and decoding outside of higher-level APIs like `MediaRecorder` or `HTMLMediaElement`.

6.1 Late 2021

WebCodecs had just reached initial availability:

- **Chrome 94** (September 2021) shipped WebCodecs as a stable feature, graduating from origin trial. Chrome 92 and 93 had offered it as an origin trial with breaking changes between versions.
- **Firefox** had no support.
- **Safari** had no support.
- The W3C specification was a First Public Working Draft (2021).

In Chrome 94, developers could for the first time:

- Encode and decode individual video frames using `VideoEncoder` and `VideoDecoder` with the browser's hardware-accelerated codecs, without being constrained by `MediaRecorder`'s real-time recording model.
- Encode and decode individual audio chunks (`AudioEncoder`/`AudioDecoder`).
- Perform faster-than-realtime encoding (previously impossible via `MediaRecorder`).

- Control per-frame keyframe insertion and quality parameters.
- Obtain raw encoded bytes without container overhead.

However, these capabilities were Chrome-only. Firefox and Safari users had no access to low-level codec APIs; they were limited to the high-level `MediaRecorder` and `HTMLMediaElement` interfaces.

6.2 March 2026

WebCodecs has achieved broad cross-browser support:

- **Chrome/Edge** — supported since version 94 (2021).
- **Firefox 130+** (September 2024) — shipped approximately three years after Chrome.
- **Safari 26+** (2024) — full support. Earlier Safari versions (16.4+) had partial video-only support.
- Approximately 94% of browser market share supports WebCodecs.
- The specification remains a W3C Working Draft (updated January 2026), not yet a Recommendation.

With cross-browser support, all major browsers can now perform frame-level codec operations using hardware acceleration. This enables web applications to build custom media pipelines — video editors, live transcoders, custom streaming clients — that were previously possible only through WebAssembly codec implementations or native applications.

7 Summary

8 Conclusion

In late 2021, the real-time web communication stack defined by these standards was in a transitional state. WebRTC was the only finalized standard with broad browser support, yet even it suffered from significant cross-browser inconsistencies — Safari's codec and feature limitations, Chrome's ongoing SDP migration, and iOS reliability issues. QUIC had just been standardized but was not yet universally enabled. HTTP/3 was shipping in browsers based on draft specifications, without a finalized RFC. WebTransport was effectively non-functional in any browser. WebCodecs had just shipped in a single browser.

By March 2026, the picture is fundamentally different. The entire protocol stack — from QUIC transport through HTTP/3 to WebRTC — is standardized via finalized RFCs and implemented by default across all major browsers. WebCodecs provides cross-browser access to hardware-accelerated codecs at the frame level. The most notable remaining gap is Safari's lack of WebTransport support, which prevents universal availability of browser-to-server unreliable datagrams and independent reliable streams. The Encoded Transforms API for WebRTC also remains Chrome-only, limiting cross-browser end-to-end encryption of media in the WebRTC pipeline.

The practical consequence is that a web application built in March 2026 can rely on a complete, standards-based real-time communication stack across all major browsers — something that was not possible in late 2021, when much of the stack was either draft, experimental, single-browser, or absent.

Standard	Late 2021	March 2026	Key Change
WebRTC 1.0	W3C Rec (Jan 2021). Chrome/Firefox full support. Safari limited: H.264 only, no simulcast, iOS DataChannel broken. Plan B still in Chrome.	Updated Rec (2024–25). Unified Plan universal. Safari has VP9. iOS reliable. JSEP updated to RFC 9429. mDNS privacy standardized.	Uneven → converged (Safari gap narrowed; Encoded Transforms still Chrome-only)
QUIC	RFC 9000 (May 2021). Chrome/Firefox default. Safari experimental, manual opt-in.	Default in all browsers (Safari 16+, 2022).	Partial → universal
HTTP/3	Pre-RFC draft implementations. Chrome/Firefox default. Safari behind flag.	RFC 9114 (June 2022). Default in all browsers. 92%+ coverage.	Draft → standard → universal
WebTransport	Experimental only. Chrome disabled (M91–94). Firefox/Safari: none.	Chrome 97+, Firefox 114+. Safari: still absent. 82% coverage.	Non-functional → broadly available (Safari gap)
WebCodecs	Chrome 94 only (Sept 2021). Firefox/Safari: none. First Public WD.	Chrome, Firefox 130+, Safari 26+. 94% coverage. Still WD.	Chrome-only → cross-browser

Table 1: Browser capability status by standard, late 2021 vs. March 2026.

9 Looking Ahead: 2028–2029 (Two to Three Years)

The following projections are grounded in active W3C charter milestones, IETF draft publication timelines, and formal browser vendor implementation signals.

9.1 WebCodecs Reaches W3C Recommendation

The W3C Media Working Group charter targets WebCodecs Candidate Recommendation in Q1 2026 and full Recommendation by Q4 2026. By 2028, the specification should be a mature, finalized standard. This means the frame-level codec access that was Chrome-only in 2021 and cross-browser by 2026 will have a stable, locked-down API surface that browser vendors commit to maintaining long-term. The WebCodecs Codec Registry will likely include formal AV1 registration entries alongside existing H.264, VP8, and VP9 entries.

9.2 WebTransport Achieves Universal Browser Support

Apple committed WebTransport as a formal Interop 2026 priority, accounting for 20% of the interoperability score. Safari already has experimental WebTransport support behind a flag in iOS 18. Given this commitment, Safari production support is expected by late 2026 or early 2027, with the W3C specification reaching Recommendation status around the same time. By 2028, WebTransport should be universally available across all major browsers — closing the last significant gap in the stack. Applications will be able to rely on browser-to-server unreliable datagrams and independent reliable streams without Safari fallbacks.

9.3 WebRTC Encoded Transforms Become Cross-Browser

The `RTCRtpScriptTransform` API (WebRTC Encoded Transform) has a Working Draft dated February 2026 and is approaching Candidate Recommendation. Safari and Firefox have implemented it natively; Chrome continues support via its earlier Insertable Streams lineage. By 2028, this API should be standardized and available across all major browsers, enabling cross-browser end-to-end encryption of media frames within the WebRTC pipeline — a capability that was Chrome-only from 2021 through 2026.

9.4 AV1 Becomes a Practical WebRTC Codec

Firefox enabled AV1 for WebRTC by default in 2025. Chrome has supported it for several years. By 2028, AV1 hardware encoding support in consumer devices will have expanded, reducing the CPU cost that currently limits its adoption. AV1's superior compression efficiency (roughly 30% better than VP9 at equivalent quality) will make it a practical default codec for WebRTC calls in Chrome and Firefox. Safari's adoption remains uncertain, as Apple historically favors H.264 with hardware acceleration.

9.5 Media over QUIC Reaches RFC Status

The IETF Media over QUIC (MoQ) transport draft (`draft-ietf-moq-transport`) is advancing rapidly, with version 15 published in October 2025. The protocol enables subscription-based media publishing and consuming over QUIC, designed for low-latency distribution with CDN support. By 2028, MoQ should have reached RFC status, along with companion specifications for media container formats (Low Overhead Media Container) and streaming formats. Browser integration will likely occur through JavaScript libraries built on WebTransport and WebCodecs rather than through dedicated browser APIs — the component APIs are already in place to support this.

9.6 RTP over QUIC Reaches RFC Status

The IETF RTP over QUIC (RoQ) draft (`draft-ietf-avtcore-rtp-over-quic`) maps RTP and RTCP packets onto QUIC streams and datagrams. By 2028, this should reach RFC status, enabling WebRTC backends to transport media over QUIC instead of the traditional UDP+DTLS-SRTP path. Browser-facing integration would occur through SDP negotiation rather than new JavaScript APIs, potentially enabling WebRTC calls to benefit from QUIC's congestion control and connection migration at the transport layer.

9.7 Hardware and Network Context

The standards projections above do not exist in a vacuum. By 2028, two underlying trends amplify what these standards make practically possible.

CPU and hardware acceleration. AV1 encoding is computationally expensive — in 2026, real-time AV1 encoding for a WebRTC call can consume an entire CPU core on mid-range hardware, which is why VP9 and H.264 remain the practical defaults despite AV1's superior compression. By 2028, dedicated AV1 hardware encoders will be standard in new laptops, phones, and tablets (Apple's M-series chips, Qualcomm Snapdragon, Intel Arc, and AMD RDNA already include or are adding AV1 encode blocks). This shifts AV1 from "technically supported but too expensive" to "hardware-accelerated and free." The WebCodecs API, which exposes hardware-accelerated codecs to JavaScript, directly benefits: an application calling `VideoEncoder` with AV1 in 2028

will hit dedicated silicon rather than a software encoder, making frame-level AV1 processing via WebCodecs practical for real-time use cases.

Similarly, the unbundled architecture — composing WebTransport, WebCodecs, and WebAssembly — imposes CPU overhead from JavaScript orchestration and WebAssembly processing that would have been prohibitive on 2021 hardware. By 2028, the roughly 40–50% improvement in single-thread CPU performance since 2021 (compounding at approximately 8–10% annually) makes this overhead manageable for real-time media pipelines.

Network bandwidth. Global median fixed broadband speeds have approximately doubled from 2021 to 2026 (from roughly 60 Mbps to over 120 Mbps in developed markets), and 5G mobile networks have expanded substantially. By 2028, continued growth in fixed and mobile bandwidth means that the efficiency gains from AV1 and QUIC compound with raw bandwidth increases. A WebRTC call using AV1 over QUIC on a 2028 connection will have both a more efficient codec (30% less bandwidth than VP9) and more available bandwidth, enabling higher resolution and frame rates within the same network cost envelope. QUIC's connection migration (RFC 9000) becomes more impactful as mobile usage grows — a video call that seamlessly survives a Wi-Fi to 5G handoff requires both the protocol support (already standardized) and sufficient mobile bandwidth to sustain the call (increasingly available).

The net effect is that the standards reaching maturity by 2028 will land on hardware and networks that can fully exploit them, rather than being bottlenecked by the infrastructure they run on.

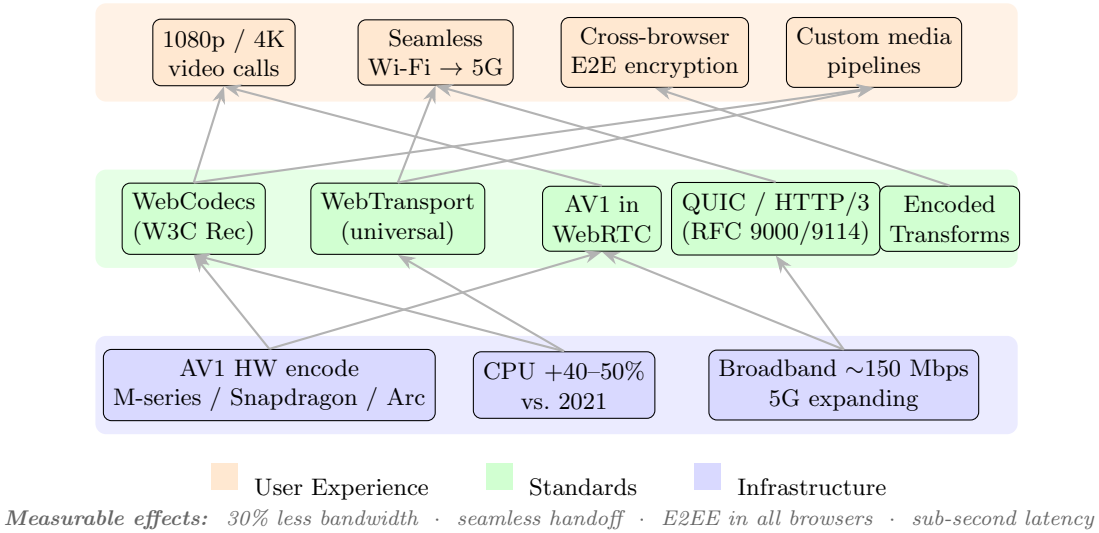

Figure 1: By 2028: infrastructure enables standards to deliver measurable user experience gains. Arrows show enabling relationships.

10 Looking Ahead: 2029–2031 (Three to Five Years)

These projections extend further from current standardization activity and carry correspondingly more uncertainty, but follow from the trajectory established by the standards work described above.

10.1 The "Unbundled" Architecture Becomes Viable

By 2029, all the component APIs — WebTransport (universal), WebCodecs (Recommendation), and RTCRtpScriptTransform (standardized) — will have been stable and cross-browser for two or more years. This makes the "WebRTC Unbundled" architecture practically viable: applications can compose WebTransport for data transport, WebCodecs for encoding and decoding, and WebAssembly for custom processing, bypassing the monolithic `RTCPeerConnection` API entirely.

This matters because the unbundled approach gives applications control over decisions that WebRTC currently makes internally: which codec to use, how to packetize, what transport to use, and how to handle congestion. Use cases that are poorly served by WebRTC's built-in assumptions — cloud gaming requiring ultra-low latency with acceptable loss, scientific visualization requiring custom frame processing, or broadcast distribution requiring one-to-many fan-out — can be built from these composable primitives.

Traditional WebRTC via `RTCPeerConnection` will persist for general video conferencing, where its built-in codec negotiation, ICE connectivity, and DTLS-SRTP security provide a well-integrated default. The unbundled approach is not a replacement but an alternative for applications with specialized requirements.

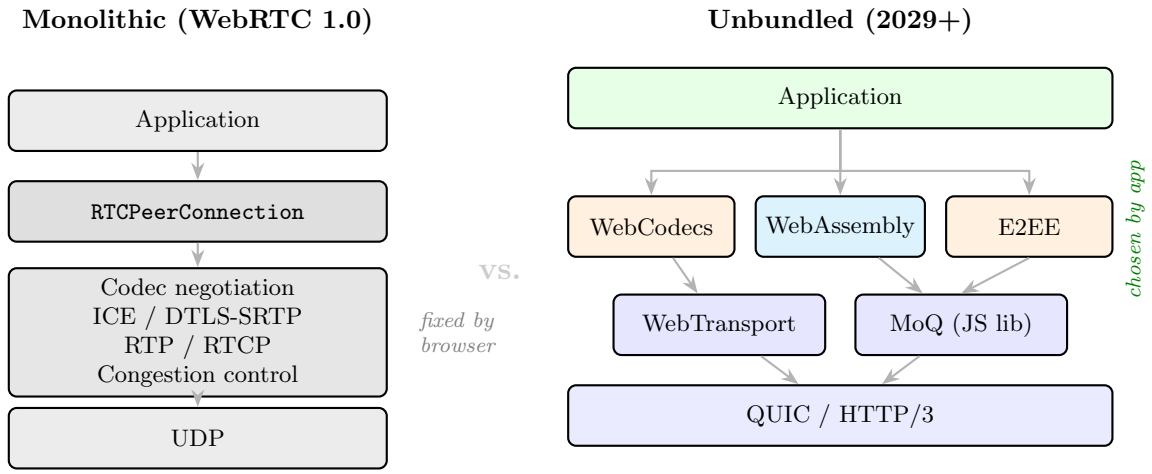

Figure 2: Monolithic WebRTC (left) vs. the unbundled architecture (right). The unbundled approach gives applications control over codec, transport, encryption, and congestion decisions that `RTCPeerConnection` handles internally.

10.2 MoQ Enables Standards-Based Low-Latency Streaming

With MoQ at RFC status and WebTransport universally available, a standards-based path exists for low-latency media streaming from server to browser that does not require WebRTC. A MoQ JavaScript library running on WebTransport and using WebCodecs for decoding could deliver sub-second latency streaming with CDN fan-out — combining the latency characteristics previously unique to WebRTC with the scalability of traditional CDN architectures. This creates a standards-based alternative to proprietary low-latency streaming protocols.

10.3 AV1 as the Default WebRTC Codec

By 2030, hardware AV1 encoding will be standard in consumer devices (laptops, phones, tablets). The CPU cost barrier that limits AV1 adoption in 2026 will have largely disappeared. AV1's compression advantage over VP9 and H.264 makes it the natural default for WebRTC, reducing bandwidth requirements for equivalent video quality. Whether Safari adopts AV1 for WebRTC depends on Apple's hardware acceleration decisions, but the broader ecosystem pressure from Chrome and Firefox — which together represent the majority of browser usage — will make AV1 the dominant codec for new WebRTC applications.

10.4 QUIC v2 Transparent Adoption

QUIC v2 (RFC 9369, published May 2023) is identical to QUIC v1 in capability but uses different wire constants to resist middlebox ossification. By 2030, browsers will have transparently migrated to QUIC v2 for new connections. This is invisible to applications — no new capabilities are exposed — but ensures the QUIC transport layer remains resilient against network intermediaries that might otherwise interfere with the protocol based on recognizing v1 byte patterns.

10.5 No WebRTC 2.0

The W3C WebRTC Working Group charter (extending to September 2026) adds new capabilities through extension specifications rather than monolithic version bumps. This pattern is expected to continue. There will be no "WebRTC 2.0" in the traditional sense. Instead, the API surface will grow incrementally through standardized extensions — Encoded Transforms, SFrame for end-to-end encryption in group calls, and future extensions not yet drafted — while the core `RTCPeerConnection` API remains stable.

10.6 Hardware and Network Context

By 2030, the hardware and network landscape further amplifies the practical impact of these matured standards.

CPU and hardware acceleration. By 2030, single-thread CPU performance will have improved roughly 60–80% relative to 2021 levels, and AV1 hardware encode/decode will be ubiquitous — present not just in flagship devices but in budget laptops and mid-range phones. This has two consequences for the standards discussed here. First, AV1 becomes the natural default codec for WebRTC: the hardware acceleration eliminates the CPU cost barrier, and WebCodecs applications calling `VideoEncoder` with AV1 will consistently hit dedicated silicon across the device spectrum, not just on high-end hardware. Second, the unbundled architecture — WebTransport plus WebCodecs plus WebAssembly — becomes practical even on resource-constrained devices. The JavaScript and WebAssembly orchestration overhead that would strain a 2021 mobile processor is routine on a 2030 device, making custom real-time media pipelines viable on phones and tablets, not just desktop machines.

Network bandwidth. If current trends hold, median fixed broadband speeds in developed markets will reach 200–300 Mbps by 2030, with 5G mobile networks providing reliable 100+ Mbps in urban areas. At these speeds, bandwidth is no longer the binding constraint for most real-time communication scenarios. The practical consequence is a shift in what matters: codec efficiency (AV1 over H.264) becomes less about fitting within tight bandwidth limits and more about enabling higher quality — 4K video conferencing, high-frame-rate screen sharing, or multi-stream layouts

— without saturating the connection. QUIC's multiplexed streams (RFC 9000) and HTTP/3's head-of-line blocking elimination (RFC 9114) matter most when multiple parallel flows compete for bandwidth, which is precisely the scenario enabled by abundant bandwidth and ambitious applications.

MoQ-based streaming benefits particularly from this environment. A MoQ library on WebTransport using WebCodecs to decode AV1 can deliver high-bitrate, low-latency streams that would have been impractical in 2021 not because the standards didn't exist, but because the combination of codec cost, network capacity, and API availability wasn't there. By 2030, all three constraints are resolved simultaneously: the standards are finalized and universal, the hardware accelerates the codecs, and the networks carry the bits.

The compounding effect. The key observation is that standards maturation, hardware improvement, and network growth are complementary, not independent. A standard that reaches browser universality on hardware that can't run it efficiently, or over networks that can't carry its output, delivers theoretical capability rather than practical impact. By the 2029–2031 timeframe, these three curves converge: every standard in this document will be finalized and universally implemented, running on hardware with dedicated acceleration for the codecs they specify, over networks with bandwidth to spare. This convergence is what makes the 3–5 year outlook qualitatively different from a simple extrapolation of today's capabilities.

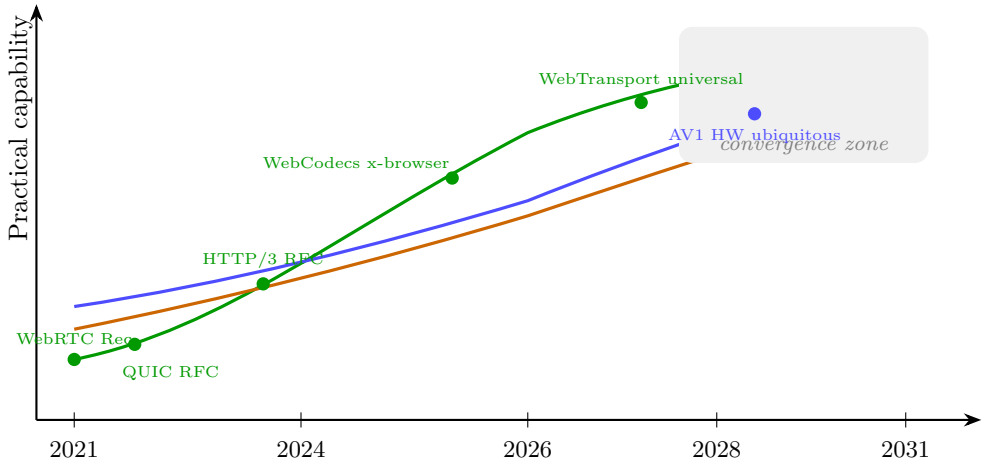

Figure 3: Convergence of standards maturity, hardware capability, and network bandwidth from 2021 to 2031. The shaded region marks where all three curves reach levels sufficient to fully exploit the standards discussed in this document.

A Information Theoretic View

This appendix applies information-theoretic measures to the real-time web communication stack described in this document. We quantify channel capacity, source coding efficiency, and effective information rates across three time horizons: today (March 2026), the near term (2027–2028), and the medium term (2029–2031). The analysis draws on the standards, codec parameters, and infrastructure projections developed in the main text.

A.1 Channel Capacity of the Transport Layer

The Shannon-Hartley theorem gives the theoretical maximum information rate for a channel with additive white Gaussian noise:

$$C = B \log_2\left(1 + \frac{S}{N}\right) \quad [\text{bits/s}] \tag{1}$$

where B is the channel bandwidth in Hz and S/N is the signal-to-noise ratio.

While the physical-layer capacity of a broadband link is governed by (1), the *effective* capacity available to the application is reduced by protocol overhead, congestion control, and transport inefficiencies. We define the effective application-layer throughput as:

$$R_{\text{eff}} = C_{\text{link}} \cdot \eta_{\text{transport}} \cdot (1 - \rho_{\text{overhead}}) \tag{2}$$

where C_{link} is the link-layer capacity, $\eta_{\text{transport}} \in (0, 1]$ is the transport efficiency factor (capturing congestion control, retransmission, and multiplexing behavior), and $\rho_{\text{overhead}} \in [0, 1)$ is the fractional protocol overhead (headers, encryption, framing).

A.1.1 Head-of-Line Blocking and Multiplexed Capacity

Consider n independent streams multiplexed over a single connection. Under TCP (HTTP/2), a single lost packet blocks all n streams. The expected throughput per stream under loss rate p is bounded by:

$$R_{\text{TCP}}^{(i)} \leq \frac{R_{\text{eff}}}{n} \cdot (1 - p)^{n-1} \tag{3}$$

because a loss in *any* of the n streams stalls all others. Under QUIC (RFC 9000), streams are independent at the transport layer, so:

$$R_{\text{QUIC}}^{(i)} \leq \frac{R_{\text{eff}}}{n} \cdot (1 - p) \tag{4}$$

The ratio of effective per-stream throughput is:

$$\frac{R_{\text{QUIC}}^{(i)}}{R_{\text{TCP}}^{(i)}} = \frac{1}{(1 - p)^{n-2}} \tag{5}$$

For $n = 8$ streams (a typical multi-track video conference) at $p = 0.01$ (1% loss), this gives a QUIC advantage factor of $1/(0.99)^6 \approx 1.062$, a modest 6.2% improvement. At $p = 0.05$ (5% loss, common on mobile or congested links), the factor is $1/(0.95)^6 \approx 1.34$, a 34% throughput advantage per stream.

A.1.2 Connection Establishment Latency

Let τ denote the one-way latency (half the round-trip time). The information-carrying capacity during connection establishment is zero; the handshake represents pure overhead. The time to first useful byte is:

$$T_{\text{TCP+TLS 1.3}} = 2\tau_{\text{TCP}} + \tau_{\text{TLS}} = 3\tau \quad \text{(1-RTT TLS 1.3)} \tag{6}$$

$$T_{\text{QUIC}} = \tau \quad \text{(1-RTT)} \tag{7}$$

$$T_{\text{QUIC, 0-RTT}} = 0 \quad \text{(resumption)} \tag{8}$$

The QUIC 0-RTT case (8) eliminates the handshake entirely for resumed connections. The information lost during handshake dead time, relative to steady-state capacity, is:

$$I_{\text{lost}} = C_{\text{link}} \cdot T_{\text{setup}} \tag{9}$$

For a 120 Mbps link with $\tau = 25$ ms (typical fixed broadband in 2026), the TCP+TLS 1.3 handshake wastes $120 \times 10^6 \times 0.075 = 9 \times 10^6$ bits (1.125 MB), while QUIC 1-RTT wastes only 3×10^6 bits (375 KB).

A.2 Source Coding Efficiency

The rate-distortion function $R(D)$ defines the minimum bit rate required to represent a source at distortion level D. For a Gaussian source with variance σ^2 under mean squared error distortion:

$$R(D) = \frac{1}{2} \log_2 \frac{\sigma^2}{D} \quad \text{[bits/sample]} \tag{10}$$

No practical codec achieves $R(D)$; real codecs operate at some rate $R_{\text{codec}} > R(D)$. We define the coding efficiency gap as:

$$\Delta = \frac{R_{\text{codec}} - R(D)}{R(D)} \tag{11}$$

The codecs in the WebRTC stack can be ordered by their empirical proximity to $R(D)$ for typical video content at conversational resolutions (720p, 30 fps). Using published Bjøntegaard Delta (BD) rate measurements:

$$R_{\text{H.264}} > R_{\text{VP9}} > R_{\text{AV1}} > R(D) \tag{12}$$

Specifically, the empirical rate savings at equivalent PSNR are approximately:

$$R_{\text{VP9}} \approx 0.65 \cdot R_{\text{H.264}} \quad (\sim 35\% \text{ savings}) \tag{13}$$

$$R_{\text{AV1}} \approx 0.70 \cdot R_{\text{VP9}} \approx 0.46 \cdot R_{\text{H.264}} \quad (\sim 30\% \text{ over VP9}) \tag{14}$$

A.2.1 Codec Entropy and WebCodecs

The WebCodecs API (W3C Working Draft) exposes per-frame codec access, allowing measurement and control of the instantaneous encoding rate. For a video frame f_t at time t, the encoder output has empirical entropy:

$$\hat{H}(f_t) = -\sum_{s \in \mathcal{S}} p(s \mid f_t) \log_2 p(s \mid f_t) \tag{15}$$

where $\mathcal{S}$ is the set of symbols emitted by the entropy coder (CABAC for H.264/AV1, arithmetic coding for VP9). WebCodecs enables applications to observe $|E(f_t)|$ (the encoded frame size in bits) directly, which serves as a practical upper bound on $\hat{H}(f_t)$:

$$\hat{H}(f_t) \leq |E(f_t)| \leq R_{\text{codec}}/\text{fps} \tag{16}$$

Parameter	March 2026	2027–2028	2029–2031
Median link capacity	120 Mbps	∼150 Mbps	200–300 Mbps
Dominant codec (WebRTC)	VP9 / H.264	VP9 / AV1 (HW)	AV1 (HW default)
Codec rate (720p/30fps)	1.5–2.5 Mbps	1.0–1.8 Mbps	0.7–1.2 Mbps
$\eta_{\text{transport}}$	0.92 (QUIC)	0.93	0.94
ρ_{overhead} (SRTP+QUIC)	0.06	0.05	0.04
Typical loss p	0.01–0.03	0.01–0.02	0.005–0.01
I_{net} (720p video)	1.3–2.2 Mbps	0.9–1.6 Mbps	0.6–1.1 Mbps
I_{net} as % of C_{link}	1.1–1.8%	0.6–1.1%	0.2–0.6%

Table 2: Effective information rate for a 720p/30fps WebRTC video stream. The declining ratio $I_{\text{net}}/C_{\text{link}}$ reflects both improved codec efficiency and growing link capacity, freeing bandwidth for higher resolutions, additional streams, or concurrent data channels.

A.3 Effective Information Rate by Time Horizon

We now combine transport and source coding measures to estimate the effective information rate for a representative real-time video session: a single 720p/30fps video call with one audio channel.

Define the *net information rate* as:

$$I_{\text{net}} = R_{\text{codec}} \cdot \eta_{\text{transport}} \cdot (1 - \rho_{\text{overhead}}) \cdot (1 - p_{\text{loss}}) \tag{17}$$

The declining ratio $I_{\text{net}}/C_{\text{link}}$ in Table 2 is the central quantitative observation: by 2030, a 720p video call consumes less than 0.5% of available link capacity. The surplus capacity enables qualitative changes — 4K video conferencing, multi-stream layouts, or concurrent data channels via WebTransport — that are quantitatively infeasible in 2026.

A.4 Security Entropy and Key Space

The DTLS-SRTP security architecture (RFCs 8827, 3711) protects all WebRTC media. Information-theoretic security requires that the key entropy exceed the information content of the protected material.

For a DTLS 1.3 handshake with ECDHE on P-256, the key agreement provides:

$$H_{\text{key}} = 128 \text{ bits of security} \tag{18}$$

The SRTP master key is 128 bits (AES-128-GCM), giving an exhaustive search cost of 2^{128} operations. The total information content of a one-hour 720p video call at 2 Mbps is:

$$I_{\text{call}} = 2 \times 10^6 \times 3600 = 7.2 \times 10^9 \text{ bits} \approx 2^{32.7} \text{ bits} \tag{19}$$

The ratio of key entropy to protected content:

$$\frac{H_{\text{key}}}{I_{\text{call}}} = \frac{128}{32.7} \approx 3.91 \tag{20}$$

This ratio remains comfortably above 1 even for much longer sessions. For a continuous 24-hour 4K AV1 stream at 15 Mbps:

$$I_{\text{stream}} = 15 \times 10^6 \times 86400 = 1.296 \times 10^{12} \text{ bits} \approx 2^{40.2} \text{ bits} \tag{21}$$

yielding $H_{\text{key}}/I_{\text{stream}} \approx 3.18$, still well above the threshold.

A.4.1 End-to-End Encryption via Encoded Transforms

The RTCRtpScriptTransform API (Encoded Transforms) enables application-layer E2EE within the WebRTC pipeline. From an information-theoretic perspective, this introduces a second encryption layer with its own key, creating a *nested channel*:

$$X \xrightarrow{K_{\text{E2EE}}} E_1(X) \xrightarrow{K_{\text{SRTP}}} E_2(E_1(X)) \tag{22}$$

The mutual information between the ciphertext and plaintext, given neither key, satisfies:

$$I(X;\, E_2(E_1(X))) \leq \min\bigl(H(K_{\text{E2EE}})^{-1},\ H(K_{\text{SRTP}})^{-1}\bigr) \cdot I_{\text{call}} \tag{23}$$

In practice, with $H(K_{\text{E2EE}}) = H(K_{\text{SRTP}}) = 128$ bits, this quantity is negligible ($\ll 1$ bit for any practical session length), confirming that the nested encryption provides information-theoretic confidentiality well beyond computational security guarantees.

Today (March 2026), Encoded Transforms remain Chrome-only. By 2027–2028 (Section 8.3), cross-browser standardization will make this nested channel universally available. By 2029–2031, SFrame for group calls will extend this to n-party sessions, where the mutual information between any non-member and the group plaintext remains negligible:

$$I(X;\, E(X) \mid K_i \notin \{K_1, \ldots, K_n\}) \approx 0 \tag{24}$$

A.5 WebTransport: Unreliable Datagrams and Channel Capacity

WebTransport (RFC 9297) introduces unreliable datagrams to browser-server communication. For latency-sensitive applications (game state, sensor telemetry), the relevant measure is not throughput but *freshness-weighted information rate*. Define the value of a datagram as exponentially decaying with delivery latency δ:

$$V(\delta) = e^{-\lambda\delta} \tag{25}$$

where $\lambda > 0$ is an application-specific decay rate. The effective information rate under unreliable delivery is:

$$I_{\text{WT}} = R_{\text{send}} \cdot (1 - p_{\text{loss}}) \cdot \mathbb{E}\Bigl[e^{-\lambda\delta}\Bigr] \tag{26}$$

Under reliable delivery (TCP/WebSocket), a retransmission delays subsequent data, so:

$$I_{\text{WS}} = R_{\text{send}} \cdot \mathbb{E}\Bigl[e^{-\lambda(\delta + p \cdot 2\tau)}\Bigr] \tag{27}$$

The ratio $I_{\text{WT}}/I_{\text{WS}}$ exceeds 1 when:

$$(1-p) \cdot e^{\lambda \cdot p \cdot 2\tau} > 1 \quad \Longleftrightarrow \quad \lambda > \frac{-\ln(1-p)}{2p\tau} \tag{28}$$

For $p = 0.02$ and $\tau = 25$ ms, this gives $\lambda > 20.2\ \mathrm{s}^{-1}$ (half-life < 34 ms). Any application where data older than ∼30 ms is useless benefits from WebTransport's unreliable mode over WebSocket — precisely the regime of interactive gaming, live cursor tracking, and real-time sensor feeds.

As of March 2026, Safari does not support WebTransport (Section 5), limiting coverage to ∼82%. By 2027–2028 (Section 8.2), universal support will allow applications to rely on (26) without fallback paths.

A.6 Projections Summary

Measure	**March 2026**	**2027–2028**	**2029–2031**
Channel capacity C_{link}	120 Mbps	150 Mbps	250 Mbps
HOL blocking factor $(1-p)^{n-2}$ at $n=8$	0.94 ($p=0.01$)	0.94	0.97 ($p=0.005$)
Source coding gap Δ	VP9: ∼0.8	AV1 HW: ∼0.5	AV1 opt: ∼0.4
I_{net} per 720p stream	1.3–2.2 Mbps	0.9–1.6 Mbps	0.6–1.1 Mbps
Max 720p streams in C_{link}	∼55–90	∼95–165	∼230–415
E2EE availability	Chrome only	Cross-browser	SFrame n-party
WebTransport coverage	82%	∼100%	100%
Freshness threshold λ^*	20.2 s^{-1}	∼18 s^{-1}	∼15 s^{-1}

Table 3: Information-theoretic measures across time horizons. Δ is the coding efficiency gap (11), I_{net} is the net information rate (17), and λ^* is the freshness decay threshold above which WebTransport dominates WebSocket (28).

The trajectory is clear: the combination of improving source coding (AV1 hardware), expanding channel capacity (broadband growth), and maturing transport protocols (QUIC universal, WebTransport universal) drives the *information cost per unit of communicated experience* steadily downward. The remaining question is not whether the Shannon capacity suffices — it already does, by orders of magnitude — but how efficiently the protocol stack converts that capacity into delivered, fresh, secure information at the application layer. By 2030, the gap between theoretical capacity and realized throughput will be the narrowest it has ever been for browser-based real-time communication.